

MeanShift: Analysis of speedup achieved through parallel programming on CPU

Emanuele nencioni

E-mail address

emanuele.nencioni@edu.unifi.it

Abstract

This report presents the implementation and performance analysis of the Mean Shift clustering algorithm for image segmentation. Its $\mathcal{O}(N^2)$ computational complexity was addressed through three sequential variants (from a baseline to SIMD-vectorized Structure of Arrays) followed by two parallelizations with OpenMP and a GPU-accelerated version using CUDA. The OpenMP variants achieve near-linear speedup ($\sim 11\times$ at 12 physical cores), while the sequential SoA layout reduces per-pixel-pair cost by $\sim 3\times$ through auto-vectorization. The CUDA implementation employs a tiled shared-memory kernel that reduces global memory traffic from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ bytes per iteration and achieves a further 14-206 $\times$ (exceeding 150x speedup on larger images) speedup over sequential variants. The report analyses the performance impact of memory layout, bandwidth, thread count, and hardware counter data across all implementations.

1. Introduction

Mean Shift[2] is a non-parametric clustering technique used for "mode-seeking" in a feature space (Figure 1). Unlike algorithms such as K-Means[3], Mean Shift does not require an *a priori* specification of the number of clusters. Instead, it discovers the underlying structure of the data by iteratively shifting each point toward the local density gradient.

In the context of image segmentation, the algorithm maps each pixel into a five-dimensional (5D) joint spatial-range domain:

$$\mathbf{x} = [r, g, b, x, y]^T$$

where (r, g, b) represents the color components and (x, y) denotes the spatial coordinates. This introduces a significant computational burden, but this high-dimensional mapping allows for sophisticated segmentation that respects both color similarity and spatial proximity.



Figure 1: left: Original image, right: mean shift clustering with bandwidth of 30, black borders.

1.1. Kernel Density Estimation and Mode Seeking

In the joint spatial-color domain, the density at a point $\mathbf{x}$ is estimated using a kernel K with bandwidth h . The Mean Shift algorithm seeks the modes of this density function by following the gradient. Mathematically, for each pixel i , we compute the Mean Shift vector $m(\mathbf{x}_i)$:

$$m(\mathbf{x}_i) = \frac{\sum_{j=1}^n \mathbf{x}_j K\left(\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{h^2}\right)}{\sum_{j=1}^n K\left(\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{h^2}\right)} - \mathbf{x}_i$$

This vector $m(\mathbf{x}_i)$ always points to the direction of the maximum increase in density. The algorithm iteratively updates $\mathbf{x}_i \leftarrow \mathbf{x}_i + m(\mathbf{x}_i)$ until the magnitude of the shift vector $\|m(\mathbf{x}_i)\|$ falls below a threshold ϵ .

Within a Brute-force approach, each of the N pixels must query all the other N in the image to calculate the weighted mean, resulting in a computational complexity of $\mathcal{O}(N^2)$. While literature often distinguishes between spatial (h_s) and range (h_r) bandwidths to fine-tune the filtering effect (Comaniciu et al. 2002 [1]), this implementation utilizes a unified bandwidth h . This choice preserves the fundamental interaction logic and the $\mathcal{O}(N^2)$ complexity while maintaining the project's primary focus on optimization and parallel efficiency rather than the heuristic tuning of clustering output.

2. language and frameworks

The implementation is developed in C++, selected for its ability to provide explicit control over memory alignment and hardware-level optimizations required for high-performance computing. The software architecture leverages SIMD Vectorization since C++ enables the use of compiler intrinsics and auto-vectorization (e.g., AVX2/AVX-512) to exploit Instruction-Level Parallelism. In this way, the 5D distance calculations can be processed as vector operations, significantly increasing the floating-point throughput per clock cycle. Parallelism is achieved through the OpenMP framework, which is uniquely suited to Mean Shift[2] due to its embarrassingly parallel nature. Since the mode-seeking trajectory of each pixel is independent, the workload allows for a data parallel decomposition. Then a CUDA (12.9) implementation is also provided to leverage the massive parallelism of modern GPUs, which can further accelerate the algorithm by distributing the computation across thousands of threads. Lastly, OpenCV is utilized as a post-processing tool for data visualization and qualitative analysis of the segmentation boundaries.

3. Methodology

This current project will see different implementations designed specifically for three-channel RGB images, focusing on the standard 5D joint spatial-range domain. For a proper performance analysis, the basic form of the algorithm was first implemented using brute-force approach with sequential computation ($\mathcal{O}(N^2)$). Then it follows a progressive optimization path to address specific bottlenecks:

1. **Refined Sequential Baseline (seq)**, An optimized version designed to remove redundant ALU overhead by pre-computing spatial coordinates.
2. **Structure of Arrays (soa)**, A memory-reorganization approach aimed at improving data locality and helping SIMD auto-vectorization.
3. **Parallel AoS (omp)**, A multi-threaded implementation utilizing OpenMP on the interleaved memory layout.
4. **Parallel SoA (omp_soa)**, A hybrid approach combining the benefits of the SoA layout with shared-memory parallelization.
5. **CUDA Implementation**, A GPU-accelerated version designed to leverage the massive parallelism of modern graphics hardware.

All the code of each implementation is available in the Appendix 7.

3.1. Baseline implementation

First, the algorithm (see appendix 7.1) computes the mean shift vector of the neighbor centered on the current pixel, with a bandwidth h . The result is a vector directed towards the highest increase in the gradient of density function. Using a uniform kernel, this vector can be represented as a vector difference of the arithmetic average (or centroid) of all points in a 5D window of a particular size h minus the current point's location. Then, each pixel's state is shifted according according to its mean shift vector to the local centroid.

From a structural perspective, the baseline source code reveals a three-level nested loop hierarchy that dictates the algorithm's execution flow and computational complexity. The **Temporal Loop** (outermost) manages the iterative refinement process; it will execute the entire mean shift pipeline until convergence or a max number of iterations is reached. Within this, the **Spatial loop** iterates through the N pixels of the images, treating each one as an independent agent seeking a local density mode. Finally, the **Neighbor loop** queries all N pixels in the image to compute the mean-shift vector and the kernel calculation. The interaction between these two inner loops generates the quadratic $\mathcal{O}(N^2)$.

This method utilizes an interleaved memory layout, which can be defined as a *logical* Array of Structures (AoS) (listing 1). Although the data are stored in a flat `std::vector<float>` rather than a formal C++ `struct`, the memory footprint is identical to an interleaved layout where all components of a single data element (the RGB channels) are stored contiguously ($[R_0, G_0, B_0, R_1, G_1, B_1, \dots]$). This often requires de-interleaving for efficient SIMD processing. In this case, this contiguous grouping provides high spatial locality for single-pixel operations, but introduces a stride-3 access pattern for individual color channels. This makes the loop difficult to auto-vectorize.

```
1 struct STBImage {
2     int width{0}, height{0}, channels{0};
3     uint8_t* rgb_image{nullptr};
4     std::string filename{};
5     ~STBImage();
6     bool loadImage(const std::string& name);
7     void saveImage(const std::string& newName)
8     const;
```

Listing 1: baseline Image implementation

3.2. seq implementation

In the initial baseline design, the spatial coordinates (x,y) of each pixel were dynamically derived from the linear index j using the image width (listing 5). While this approach minimizes the memory footprint by avoiding the storage of redundant spatial data, it shifts the primary bottleneck to the

CPU's execution units. Integer division (/) and modulo (%) are among the highest-latency scalar instructions on modern x86 architectures. Performing these operations N^2 times per iteration can cause the inner loop to become heavily **compute-bound**. To prioritize execution efficiency, in this new implementation, spatial coordinates are pre-computed and stored within the feature vector (listing 2). This expansion of the 'pixel' data structure effectively trades memory capacity for a significant reduction in clock cycles per iteration, transforming the algorithm into a more efficient memory-bound process. As a result, the initial `stb_image` buffer is replaced by a tailored data layout.

Another improvement involved **memory hoisting**, implemented by moving the auxiliary buffer allocations outside the **temporal loop**. This can reduce the overhead of the memory manager by avoiding $O(\text{iter})$ system calls and heap allocations. This was maintained across the subsequent implementations described in this section.

```
1 struct Pixel {
2     float r, g, b;
3     float x, y;
4 };
```

Listing 2: seq spatial save into pixel struct

3.3. soa implementation

In the previously described AoS layout, each pixel is represented by a contiguous block of five floating-point values. Although this ensures locality for individual pixel updates, it introduces a stride-5 access pattern when the algorithm evaluates a specific dimension across the dataset. This non-contiguous access can be suboptimal as it complicates the "gathering" of data into vector registers.

To mitigate these limitations, the Structure of Arrays (SoA) implementation reorganizes the memory layout by decomposing the feature vectors into five independent contiguous arrays (V_R, V_G, V_B, V_x, V_y), each of size N (listing 11). This architectural shift provides two primary advantages:

1. **SIMD Vectorization**, the SoA layout is inherently "SIMD-friendly." Because values of the same dimension are stored contiguously, the compiler can leverage SIMD extensions, to load and process eight pixels simultaneously in a single 256-bit register. This can eliminate the need for expensive shuffle or de-interleaving instructions required by the AoS layout.
2. **Unit-Stride Memory Access**, by ensuring that data are accessed with a stride of one, the implementation can maximize cache-line utilization. When the processor fetches a cache line, it retrieves multiple relevant values for a single dimension, reducing the total number of memory requests and improving overall throughput.

The transition from the interleaved `stb_image` format to this SoA representation is handled during the initialization phase.

3.4. omp implementation

In this other implementation an attempt to parallelize the seq implementation was made trying to achieve even more performance boost. The analysis started from the identification of the optimal level of parallelism on each loop: the **temporal loop**, the external one, is not parallelizable due to temporal dependencies (the new iteration depends on the previous one), the **neighbor loop** (innermost) would trigger a "fork-join" operation, for example, given an image with a standard resolution, this would result in hundreds of thousands of thread synchronization per second, leading to massive overhead that would dwarf the actual computation. The **Spatial loop** is the answer. Each thread is assigned to a contiguous block of pixels to process independently.

3.4.1 Reduction and thread safety

The algorithm must track the maximum shift magnitude across all pixels in order to determine convergence. In the sequential environment this is straightforward (`if (change > max_change) max_change = change;`) but in a parallel environment there are multiple threads attempting to update 'max_change' generating a **race condition**. To resolve this, the implementation uses the OpenMP **reduction** clause (listing 3). This allows each thread to maintain a local and private maximum within its own registers or L1 cache. Only upon termination of the parallel region, OpenMP performs a single thread-safe comparison of these local values (typically via a tree-based reduction) to update the global variable.

```
1 #pragma omp parallel for reduction(max:
   max_change)
2 for(int i = 0; i < n_pixels; ++i) {
3     // ... computation ...
4     if (change > max_change) max_change = change;
5 }
```

Listing 3: Parallel decomposition of the spatial loop utilizing OpenMP max-reduction for thread-safe convergence tracking.

3.4.2 Memory hoisting and Jacobi semantics

While memory hoisting was introduced in the seq implementation, it becomes fundamental to the performance of this one. In a multi-threaded environment, leaving the auxiliary buffer outside avoids **thread starvation** where worker threads can remain idle while the master thread performs the sequential task of allocating a new vector. In this way, the

implementation reduces the serial overhead between successive iterations, maximizing core utilization.

The use of separate "current" and "next"(listing 4) buffers enforces **Jacobi semantics**, which provides the mathematical foundation for thread safety. This also allows the **spatial loop** to strictly adhere to **Bernstein's conditions** for parallel executions:

- **flow and anti-dependence** (RAW, WAR) are avoided by ensuring that threads only read from the *current* and only write on the *next* buffers,
- **output dependence** (WAW) is deleted since each thread is assigned a unique pixel index i in the output array, removing the risk of multiple threads attempt to write to the same memory address.

Consequently, the algorithm removes **READ-AFTER-WRITE (RAW)** hazards without the need for costly synchronization primitives, such as *atomic* or *critical* sections. This ensures that the parallel version of the means shift algorithm [2] is bit-wise identical to the sequential version.

```

1 // Memory Hoisting: Buffer allocated once
  // outside the temporal loop
2 std::vector<Pixel> next(n_pixels);
3
4 for(; iter < max_iter; ++iter) {
5     #pragma omp parallel for schedule(
6         static)
7     for(int i = 0; i < n_pixels; ++i) {
8         // ... compute new_pixel based on "
9         // current" snapshot ...
10        next[i] = new_pixel; // Write to
11        // auxiliary buffer
12    }
13
14    // Jacobi Semantics: O(1) pointer swap
15    // to prepare for next iteration
16    current.swap(next);
17 }
```

Listing 4: Structural implementation of buffer hoisting and Jacobi double-buffering.

3.5. *omp_soa* implementation

The final refinement of the algorithm integrates the Structure of Arrays (SoA) memory layout with OpenMP multithreading. This version represents the most optimized form of the Mean Shift implementation, as it addresses both computational throughput and memory bandwidth efficiency.

3.5.1 Data locality and thread efficiency

In the standard OpenMP implementation using an AoS layout, each thread must load interleaved R, G, and B values

into registers. By switching to SoA, each thread now accesses contiguous blocks of memory for each color channel. This enables significantly improves the cache hit rate during the parallel spatial loop.

The spatial locality is maximized due to the separation of the feature space components into large, independent arrays. This reduces memory bandwidth bottlenecks and keeps the execution units saturated.

As previously established, the SoA layout is a prerequisite for efficient SIMD operations; consequently, this contiguous memory access enables each individual core to directly leverage vector instructions.

```

1 float xj = static_cast<float>(j % width);
2 float yj = static_cast<float>(j / width);
```

Listing 5: baseline spatial indexing

3.6. CUDA implementation

On GPU architectures, memory coalescing is critical for throughput. A warp of 32 threads executes one global memory instruction at a time; if all 32 addresses fall within the same 128-byte cache line, the hardware issues a single coalesced transaction. With an AoS layout, each thread reads a 20-byte struct — the warp's addresses span 5 cache lines, requiring 5 transactions. With SoA, each thread reads a consecutive float — all 32 addresses fit in one cache line. The CUDA implementation therefore stores the five feature channels as separate device arrays, matching the SoA layout of section 3.3. Conversion from the host AoS representation happens once at load time; all iteration data resides on the device. The **Spatial Loop** is mapped to the CUDA grid (one thread per output pixel) following the same reasoning as the OpenMP parallelization (section 3.4).

3.6.1 Tiled shared-memory

Without tiling, each of the N threads reads all N pixels from global memory every iteration — N^2 global loads (e.g. 68K pixels and 5 channels this reaches 93 GB per iteration). With a tiling strategy, each block of B threads cooperatively loads one tile of B pixels from global memory into shared memory (coalesced), then all B threads consume that tile from shared memory (~ 5 cycles) instead of global memory (hundreds of cycles). Over $\lceil N/B \rceil$ tiles, each pixel is loaded from global memory exactly once per iteration, reducing traffic from $O(N^2)$ to $O(N)$ bytes.

The kernel structure is shown in Listing 12. After the tile loop, each thread computes the new pixel value from its accumulated sums and writes it to the output arrays. Convergence tracking uses a float-as-uint atomicMax, relying on the property that IEEE 754 non-negative floats sort identically to their bit patterns.

Component	Specification
OS	Arch Linux (kernel 6.19.14)
Compiler	GCC 16.11 (C++17 standard)
Optimization Flags	-O3 -march=native
Parallel runtime	OpenMP (bundled with GCC)
Memory	32 GiB DDR5 RAM

Table 1: Benchmark workstation spec’s summary

3.6.2 Block size selection

The block size B controls the shared-memory footprint per block ($5 \times B \times 4$ bytes) and the number of tiles per iteration ($\lceil N/B \rceil$). Supported values are $B \in \{128, 256, 512\}$; $B = 256$ is the default, expected to provide the best balance between shared memory utilization and tile-loop overhead.

Regarding the grid dimensionality, a 2D block mapping is typically beneficial for spatially-local algorithms (e.g., convolution), where it improves cache utilization and boundary control. In this algorithm, however, every pixel is processed independently regardless of its spatial position — the 2D structure offers no exploitable locality. The additional complexity of non-square boundary handling therefore outweighs any potential benefit, making the 1D grid the preferable choice.

4. Testing Environment and Methodology

The experiments were conducted on a workstation featuring the AMD Ryzen 9 9900X processor, based on the Zen 5 architecture. This CPU provides 12 physical cores (24 logical threads via SMT) with a 64 MiB shared L3 cache. In table 1 are summarized more software and hardware specifications. 32 GB of RAM and a Nvidia GeForce GTX 1060 with 6GB of VRAM. The experimental evaluation was conducted on a workstation featuring an AMD Ryzen 9 9900X processor. This CPU integrates 12 physical cores with 24 logical threads via Simultaneous Multithreading (SMT) and features a 64 MiB shared L3 cache. The system is also equipped with 32 GB of main memory and an NVIDIA GeForce GTX 1060 GPU with 6 GB of VRAM. A comprehensive summary of the hardware and software environment is provided in table 1.

4.0.1 Clock Selection and Benchmarking Methodology

For all performance measurements, `std::chrono::steady_clock` was used instead of `std::chrono::high_resolution_clock`. While the latter suggests higher precision, the C++ standard does not guaranty its steadiness; on some implementations, it may alias the system wall-clock, making it susceptible to timing discontinuities (e.g. NTP adjustments). In contrast,

`steady_clock` is guaranteed to be monotonic, ensuring that elapsed time measurements remain consistent and reproducible across different execution environments. On the Ryzen 9900X test platform, this clock provides nanosecond resolution, which is more than sufficient for capturing the micro-scale performance in the different implementations.

4.0.2 Dataset and benchmark setup

The testing methodology was rigorously designed to evaluate the scalability and architectural efficiency of the various Mean Shift[2] implementations. This isolates computational workload (pixel count) from convergence complexity (image texture and gradients)

4.0.3 Dataset

Initial testing utilized synthetic inputs, such as solid-color and flat-gradient images (e.g., `solid_100.png` or `red400.png`). However, these were discarded because they typically converge in a single iteration, failing to provide a realistic workload for correct profiling.

The standard Berkeley Segmentation Dataset (BSD500) was utilized to ensure that the algorithm operates in a regime requiring multiple iterations, essential for accurate speedup and parallel efficiency profiling. These natural photographs contain smooth gradients, textured regions, and overlapping color distributions that provide a realistic stress test for the Mean Shift[2]. In particular, the benchmark employs a 5×5 cross-validation grid yielding 25 test configurations. Five distinct source images, ranging from "low" to "very high" complexity, are evaluated across five geometric resolution tiers (ranging from 6,700 to 68,480 pixels). By running all 5 source images at each of the 5 resolution tiers, execution times can be averaged across sources. This yields a tier-representative mean that isolates the scaling impact of pixel count without being biased by the convergence behavior of any single image.

4.0.4 Benchmarking Parameters

Parallel scaling analysis requires speedup ($S = T_1/T_p$) to be defined over a strictly fixed amount of work. Consequently, the benchmark deliberately caps the convergence loop at a maximum of 5 iterations (`MAX_ITER = 5`). This configuration explicitly isolates the per-iteration cost of the algorithm’s inner loop rather than measuring stochastic total convergence time. Furthermore, the benchmark stresses the micro-architecture by varying the spatial-color bandwidth parameter ($h \in \{20, 50, 100\}$): $h = 20$ produces tight clusters and short ranges, demanding many iterations and yielding high compute intensity per iteration; $h = 100$ instead, create large neighborhoods, forcing broad spatial scans that

test the system’s transition toward memory-bandwidth saturation.

Additionally, for every single test configuration (image, algorithm, and thread count), an initial unmeasured run is executed to warm up the hardware state. This preliminary execution ensures that the L1/L2 data and instruction caches are populated, branch predictors are primed, and operating system demand-paging overheads are resolved. Consequently, the subsequent timed iterations strictly measure the steady-state algorithmic performance, preventing initialization latency from skewing the microarchitectural profiling.

5. Result and discussion

This section presents the performance evaluation of the four Mean Shift implementations. By sweeping across a multi-dimensional benchmark matrix, we analyze how the architectural optimizations discussed in section 3 translate into real-world speedups on modern consumer hardware. TALK HERE ABOUT perf.

5.0.1 The kernel problem

At first, a runtime-selectable kernel was implemented to test each one within the benchmark. This outlined a problem: SIMD operations can only be utilized when there is no variability in this function. Since the kernel is chosen at runtime, all the benchmark resulted in worst cases possible using SoA. Given that the project doesn’t have to focus on the kernel quality of the segmentation results, It was decided to hard code the flat kernel. A deeper analysis of this problem can be found in the Appendix(fig. 11).

5.1. SoA vs seq vs baseline

The empirical results corroborate the architectural design choices detailed in section 3. As shown in fig. 2, the performance hierarchy remains consistent across all resolution tiers, with the SoA variant providing the most significant reduction in execution latency. The substantial time reduction observed for the SoA variant over the Baseline validates the transition to unit-stride memory access discussed in section 3.3. While the seq(AoS) version reduces ALU pressure by pre-computing coordinates, the SoA version successfully mitigates the “cache pollution” inherent in interleaved data structures, allowing the hardware to maximize throughput. Is important to notice the near-identical performance in varying bandwidth parameters (h). This will be explored further in section 5.1.1. Lastly, the absolute gap between baseline and optimized variants (seq and soa) widens at larger resolutions, a natural consequence of all three sharing the same $\mathcal{O}(n^2)$ scaling multiplied by different per-pixel constants. This confirms that the overhead of on-the-fly coordinate recomputation (listing 5) dominates

the execution time as N grows, while the optimized versions (section 3.2 & section 3.3) maintain a more favorable scaling curve by shifting the workload to vectorizable floating-point operations.

Speedup Looking at fig. 3, the speedup of the SoA implementation over the Baseline is approximately $2.9\times$ across all resolution tiers. This flatness is due to the cancellation of the $\mathcal{O}(n^2)$ term, leaving only the ratio of per-pixel-pair costs $C_{baseline}/C_v$, where $v = seq, soa$, and C captures the average time spent evaluating one (i, j) pair: memory loads, distance computation, kernel comparison and conditional accumulation (see Appendix 7.6 for full breakdown). This can be seen as an algorithmic speedup. The minimal gain of the seq ($\times 1.1$) is due to making the coordinate recomputation nearly free.

5.1.1 Some deeper analysis

Let’s look deep into the fig. 2, what maybe we could expect is the theoretical increase in the spatial search range when $h = 100$, but the execution time does not scale linearly with the search volume. The $\mathcal{O}(N^2)$ 5D distance computation dominates (95-98% of cycles), and the bandwidth only controls the branch-taken rate of the accumulator (2-5%) this because the brute force inner loop iterates over all N pixels regardless of bandwidth, computing the full 5D distance. The bottleneck is therefore the quadratic distance computation (flat kernel), confirming the parallelization strategy is correctly targeted. The perf hardware-counter data sheds further light on how each memory layout achieves its throughput at the microarchitectural level. As shown in table 2, the per-instruction metrics produce a counter-intuitive picture: the soa implementation exhibits a higher L1 D-cache miss rate and a lower IPC than both seq and baseline. Taken in isolation, these figures would suggest that the SoA memory layout is *detrimental* to cache behavior. However, inspecting the raw counter values reveals that this interpretation is incorrect. As fig. 9a shows, the *absolute* number of L1 D-cache misses is virtually identical across all three implementations ≈ 294 M). The elevated miss *rate* is therefore an arithmetic artifact rather than a sign of additional cache pressure. The root cause is visible in figs. 9b and 9c: the soa implementation issues approximately $3\times$ fewer load operations and retires $3\times$ fewer instructions than seq. Since the L1 miss rate is defined as misses/loads, a threefold reduction in the denominator inflates the rate even when the numerator stays constant. The same effect depresses the IPC figure, because IPC is similarly normalized per retired instruction rather than per unit of useful work. This behaviour is consistent with the compiler auto-vectorising the SoA inner loop using SIMD instructions (AVX2 on the test machine), each of which pro-

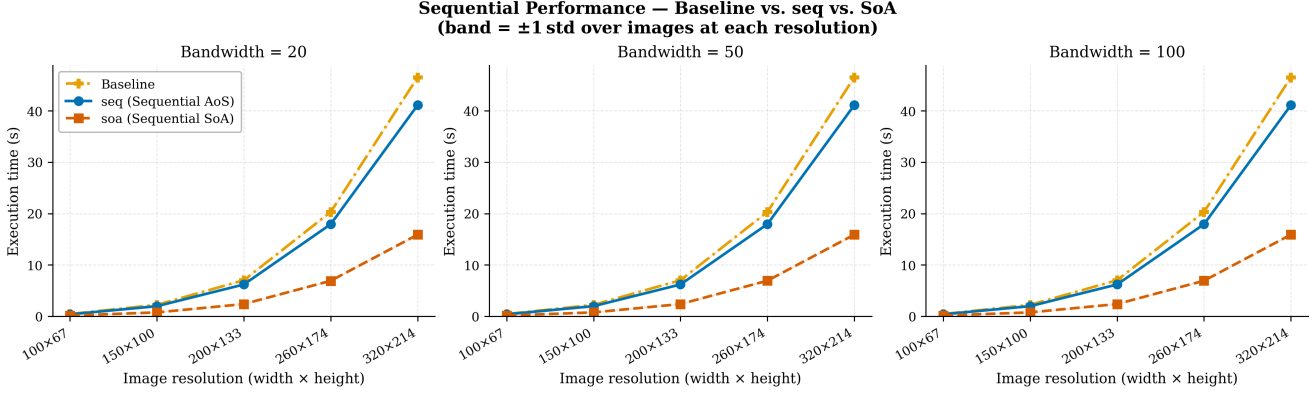


Figure 2: Structure of Arrays (SoA) exhibits superior cache locality and throughput compared to Array of Structures (AoS) and Baseline, achieving a 2.8 $\times$ reduction in execution latency at higher resolutions. The invariance of performance across bandwidth settings (20–100) suggests a compute-bound or cache-latency-bound bottleneck rather than a memory bandwidth limitation. Shaded regions denote ± 1 standard deviation across diverse image sets.

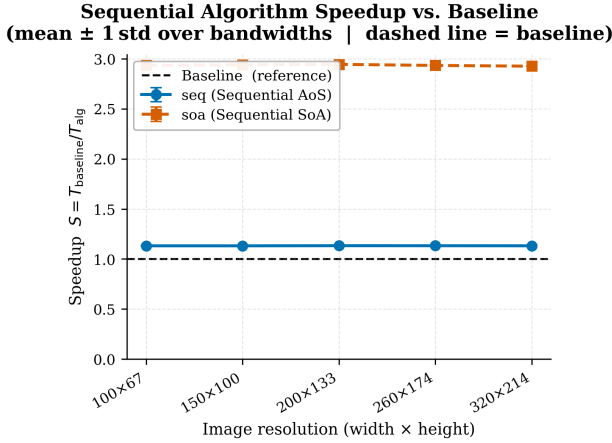


Figure 3: The constant ratio confirms a fixed per-pixel efficiency gain independent of image size.

cesses multiple pixels simultaneously and therefore counts as a single load and a single retired instruction in the hardware counters. The direct consequence is visible in fig. 4: soa completes in approximately 2.9 $\times$ fewer CPU cycles than seq, confirming that the SIMD widening translates directly into a proportional reduction in execution time. In summary, the per-instruction metrics (IPC, L1 miss rate) are misleading when SIMD is involved, because their denominators collapse with instruction count. The correct metrics to compare implementations under vectorisation are the *absolute* counters — total cycles, total instructions, and total stall slots — all of which show soa as the clear winner. The cycle-level data confirms that this workload is bound by L1 cache hit latency and instruction retirement through-

put rather than main-memory bandwidth — consistent with the bandwidth invariance analysis above.

Table 2: Sequential implementations profiling comparison

Metric	Baseline	seq (AoS)	soa (SoA)
IPC	2.80	3.03	2.36
L1 miss rate	9.0%	12.2%	30.8%
LLC miss rate	0.48%	0.50%	0.46%
Backend stalls/cycle	4.98	4.71	5.06

5.2. Parallelization with OpenMP

Unlike the sequential optimizations, where the speedup was a constant factor independent of problem size, the parallel speedup introduces dependencies on both image resolution and bandwidth (fig. 10). Larger images achieve higher parallel efficiency, as the $\mathcal{O}(n^2)$ workload amortizes the fixed serial overhead of thread synchronization and reduction. The bandwidth, negligible in the sequential analysis, now has a measurable effect — particularly in the SoA-based parallel variant. This section examines each factor in turn, then discusses the scaling behavior with simultaneous multithreading.

Resolution effect Speedup improves with image size - larger n means the $\mathcal{O}(n^2)$ parallel work grows while serial overhead (barrier, reduction and swap) stays constant per iteration. At the smaller tier, the serial fraction is measurable; at the largest it’s negligible, and speedup approaches the number of physical cores ($\approx 11\times$ at 12 threads).

layout experiment | bw = 50 | 1 thread | error bars = ± 1 std over perf runs

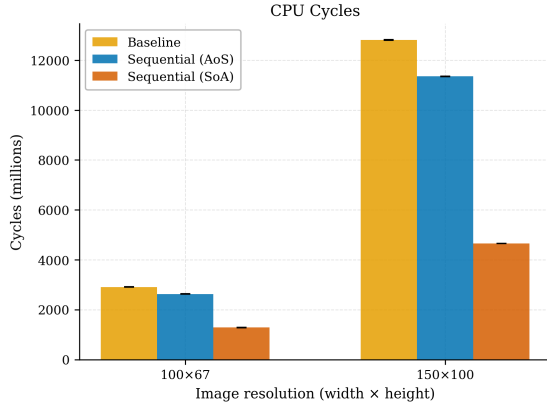


Figure 4: Total CPU cycles consumed by each sequential implementation (per iteration). This is consistent across bandwidth settings, reinforcing the conclusion that the workload is bound by L1 cache latency and instruction retirement throughput rather than memory bandwidth. Shaded regions denote ± 1 standard deviation across diverse image sets.

Table 3: Execution time (shift_ms) at 1 thread across bandwidth.

Bandwidth	omp (AoS)	omp_soa (SoA)
20	41.6	33.5
100	41.6	42.7
200	41.5	52.4
500	41.5	53.6
1000	41.5	53.7

Bandwidth effect For `omp`, bandwidth has no measurable effect. speedup is $\approx 11\times$ regardless of bandwidth (table 3). Same as sequential. For `omp_soa`, the store is more nuanced: the absolute time at 1 thread varies with bandwidth (≈ 33 s at bw=20 vs 54 s at bw=1000), but the speedup ratio remains similar. This suggests the bandwidth affects the per-pixel-pair constant C in the SoA layout but not the parallel efficiency itself.

SMT Marginal gain from 12 to 24 threads ($\approx 11\times$ to $\approx 11.5\times$) confirms the workload is compute-bound on physical cores; hyperthreading only helps hide remaining pipeline stalls rather than adding throughput (see §section 7.7 for the full hardware analysis).

As shown in Table 4, the SoA SIMD advantage only pays off at small bandwidths. At large bandwidths, the extra instruction count and memory stream overhead dominate, making `omp` faster despite its better IPC and cache

Table 4: The SoA layout wins at small bandwidth but loses at large bandwidth, where the $\sim 14\%$ (fig. 12) instruction-count overhead dominates.

Scenario	omp	omp_soa	Faster
1T, bw=20	41.6 s	33.5 s	omp_soa
1T, bw=1000	41.5 s	53.7 s	omp
12T, bw=20	3.75 s	3.27 s	omp_soa
12T, bw=1000	3.74 s	5.14 s	omp

behavior. Overall, the parallelization achieves near-linear speedup ($\sim 11\times$ at 12 cores), confirming the workload is embarrassingly parallel, while the choice of memory layout interacts with bandwidth in ways that do not affect parallel efficiency.

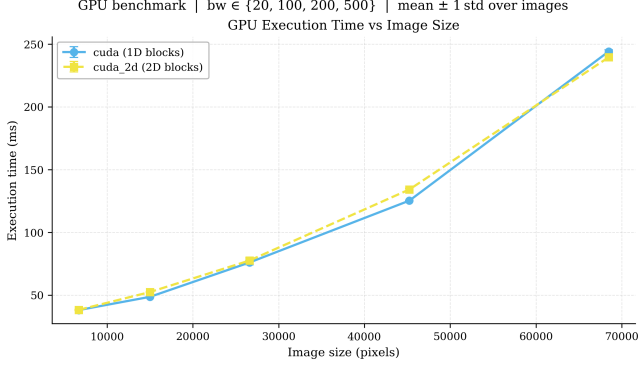
5.3. CUDA

The CUDA implementation was evaluated on the same benchmark matrix as the CPU variants. Numerical reproducibility across implementations was verified for the benchmark configuration: with a fixed number of iterations (`max_iter=5`) and Jacobi semantics, all variants produce identical output images. Under full convergence to a tolerance, floating-point summation order could theoretically cause minor divergences near the rounding boundary (section 6).

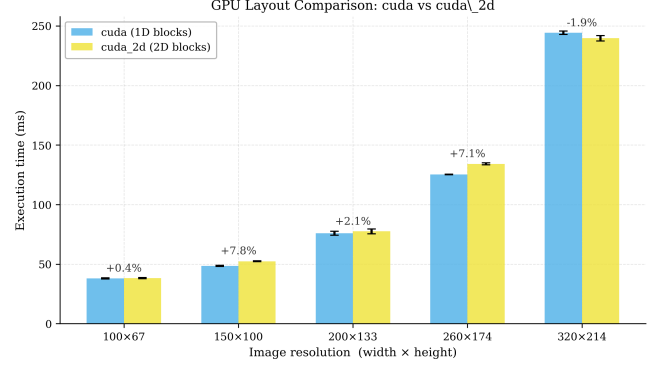
Performance-wise, the GPU results are shown in fig. 5. A 2D kernel variant was also evaluated, but its negligible advantage confirms that the 2D grid offers no exploitable locality benefit: every pixel independently scans all N others, so thread position in the image does not affect memory access.

speedup fig. 6 shows the speedup of the CUDA implementation over the sequential and OpenMP variants. On the largest test images (tier 5, 68K pixels), the GPU reaches $203\times$ speedup over `seq` and $17\times$ over the best OpenMP variant. The speedup grows with image size: small images (tier 1, 6.7K pixels) underutilize the GPU, with shift times of only ~ 3 ms per iteration, while larger images saturate the device’s throughput.

sensitivity analysis The kernel breakdown (Figure 8) separates execution time into three components: `convert_ms` (host conversion between AoS uint8 and SoA float, plus H2D and D2H PCIe transfers), `shift_ms` (pure GPU kernel computation, measured via CUDA events), and the remaining overhead. For small images, `convert_ms` dominates — the GPU is underutilized, limiting speedup. For large images, `shift_ms` dominates, confirming the GPU



(a) GPU Time vs Pixels



(b) GPU Layout Comparison

Figure 5: GPU execution time vs pixel count (left) and comparison between 1D and 2D grid layouts (right). At small resolutions the 1D grid is slightly faster due to simpler boundary handling; at large resolutions the 2D grid matches or marginally outperforms the 1D variant (1.5%), as the spatial decomposition better utilizes the GPU’s scheduling resources.

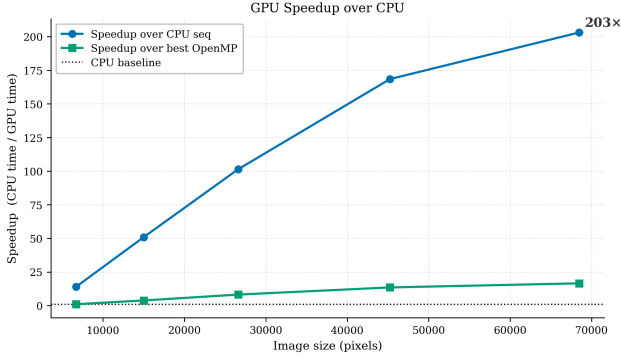


Figure 6: GPU speedup over the best CPU variant (SoA) increases with image size, reaching up to $\sim 203\times$ at the largest tier, reflecting the GPU’s ability to amortize the $\mathcal{O}(n^2)$ workload across thousands of cores.

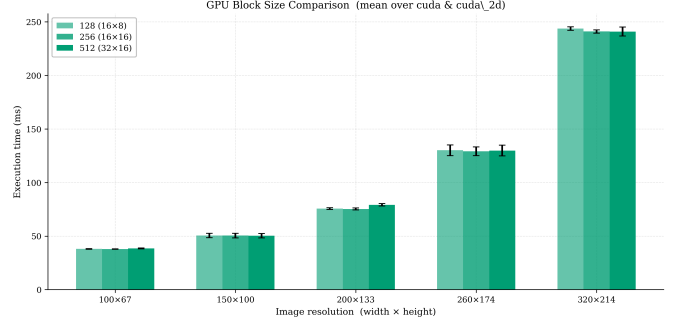


Figure 7: Execution time for block sizes 128, 256, and 512, averaged across the 1D and 2D variants (2D variants sizes in parenthesis). All three are nearly identical, confirming occupancy is not the bottleneck.

is fully saturated and explaining why kernel launch overhead and block size have negligible impact. For large images, the shift computation dominates, confirming the GPU is fully saturated and explaining why block size has negligible impact, as shown in fig. 7). This compares execution times for block sizes 128, 256, and 512. All three produce nearly identical results, confirming that occupancy is not a bottleneck for this workload — the algorithm is bound by memory latency and arithmetic throughput rather than thread count. The GPU also exhibits bandwidth invariance consistent with the sequential analysis: execution time is independent of bandwidth for both the 1D and 2D variants, as expected for a brute-force $\mathcal{O}(n^2)$ algorithm where the inner loop always scans all N pixels regardless of bandwidth. Additionally, the tile benchmark (without shared memory) shows that the tiling strategy provides approximately $2\times$

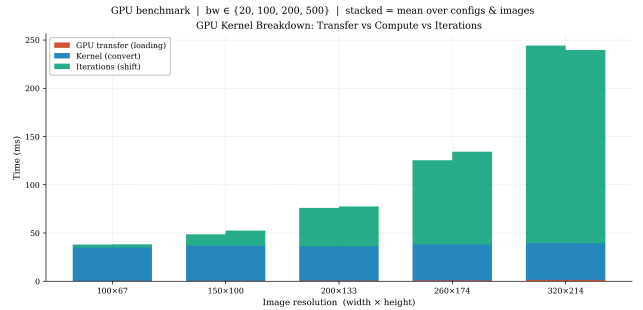


Figure 8: For each image resolution, left bar is the 1D variant (*cuda*) and the right is the 2D one (*cuda_2d*).

speedup over a direct global-memory kernel on large images, validating the shared-memory optimization discussed in section 3.6.

6. Conclusion

Numerical precision limitations It should be noted that numerical reproducibility across implementations is not guaranteed. While Jacobi semantics ensures all variants read from the same snapshot at the start of each iteration, the accumulation of floating-point summations in different orders (sequential, OpenMP reduction, CUDA atomicMax) introduces path-dependent rounding errors. Over multiple iterations, these differences can compound and, in pixels near the uint8 rounding boundary, may produce divergent output values. For the benchmark configuration (max_iter=5), the differences are negligible; for full convergence, they could theoretically shift a small fraction of pixels by one uint8 level.

Future work Future work can be focused on improving numerical consistency across parallel variants, or adopting a deterministic reduction strategy for the convergence check to guarantee identical iteration counts.

References

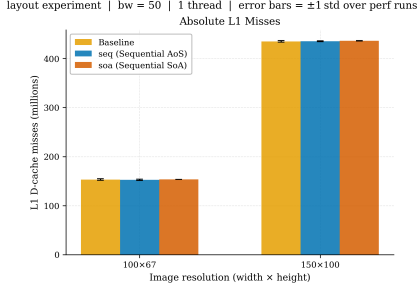
- [1] D. Comaniciu and P. Meer. Mean shift: a robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002.
- [2] K. Fukunaga and L. Hostetler. The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on Information Theory*, 21(1):32–40, 1975.
- [3] J. MacQueen. Multivariate observations. In *Proceedings of the 5th Berkeley symposium on mathematical statistics and probability*, volume 1, pages 281–297. University of California press Oakland, CA, USA, 1967.

7. Appendix

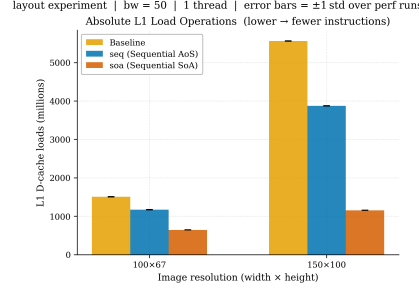
7.1. The Baseline algorithm

```
1 meanShiftResult meanShiftBaseline(STBImage& image
2   , float bandwidth,
3   int max_iter,
4   float tol, bool show_pbar, KernelFn kernel) {
5   using clock = std::chrono::steady_clock;
6
7   if(!kernel) kernel = makeKernel("flat");
8
9   const int width = image.width;
10  const int n_pixels = image.width * image.height;
11  uint8_t* raw = image.rgb_image;
12
13  // Load raw uint8_t* into float working
14  // buffer
15  std::vector<float> current(n_pixels * 3);
16  for(int i = 0; i < n_pixels * 3; ++i)
17    current[i] = static_cast<float>(raw[i]);
18
19  const float bandwidth_sq = bandwidth *
20    bandwidth;
```

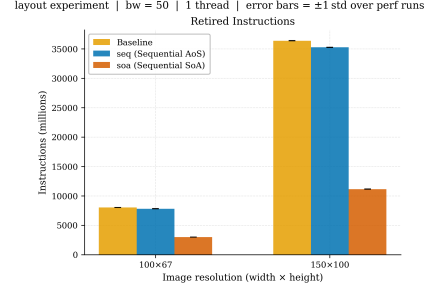
```
17 double total_shift_ms = 0.0;
18 int iter = 0;
19 std::vector<IterationInfo> iter_details;
20
21 for(; iter < max_iter; ++iter) {
22   std::vector<float> next(n_pixels * 3);
23   float max_change = 0.0f;
24
25   auto t_shift_start = clock::now();
26
27   for(int i = 0; i < n_pixels; ++i) {
28     float sum_r = 0.0f, sum_g = 0.0f,
29     sum_b = 0.0f;
30     float weight_sum = 0.0f;
31
32     for(int j = 0; j < n_pixels; ++j) {
33       float w = kernel(
34         squaredDistanceBaseline(current, i, j, width)
35         , bandwidth_sq);
36       if(w > 0.0f) {
37         sum_r += w * current[j * 3 +
38         0];
39         sum_g += w * current[j * 3 +
40         1];
41         sum_b += w * current[j * 3 +
42         2];
43         weight_sum += w;
44       }
45     }
46
47     float change = 0.0f;
48     if(weight_sum > 0.0f) {
49       float new_r = sum_r / weight_sum;
50       float new_g = sum_g / weight_sum;
51       float new_b = sum_b / weight_sum;
52       change = std::max({std::abs(new_r
53         - current[i * 3 + 0]),
54         std::abs(new_g
55         - current[i * 3 + 1]),
56         std::abs(new_b
57         - current[i * 3 + 2])});
58       next[i * 3 + 0] = new_r;
59       next[i * 3 + 1] = new_g;
60       next[i * 3 + 2] = new_b;
61     } else {
62       next[i * 3 + 0] = current[i * 3 +
63       0];
64       next[i * 3 + 1] = current[i * 3 +
65       1];
66       next[i * 3 + 2] = current[i * 3 +
67       2];
68     }
69
70     if(change > max_change)
71       max_change = change;
72   }
73
74   auto t_shift_end = clock::now();
75   double iter_ms = std::chrono::duration<
76   double, std::milli>(t_shift_end -
77   t_shift_start).count();
78   total_shift_ms += iter_ms;
79   iter_details.push_back({iter + 1, iter_ms
80   , max_change});
81
82   current.swap(next);
```



(a) L1 Miss Count



(b) L1 Loads



(c) Instructions

Figure 9: Comprehensive profile metrics including L1 cache misses, total loads, and instruction counts.

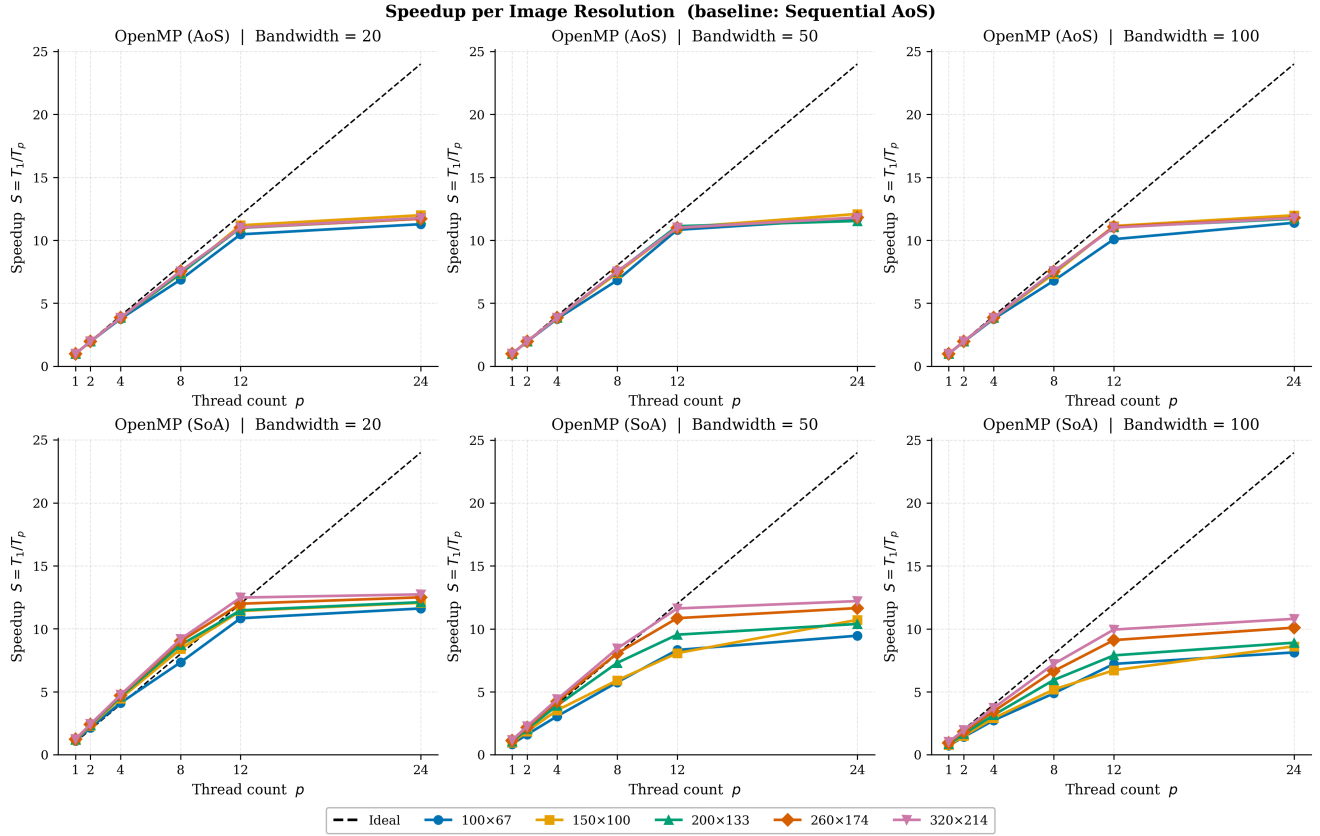


Figure 10: The near-linear scaling at 12 cores ($\approx 11\times$) confirms efficient parallelization, while SMT (24 threads) provides marginal additional benefit ($\approx 11.5\times$).

```

69
70     if(show_pbar)
71         printProgressBar(iter + 1, max_iter,
72             max_change);
73
74         if(max_change <= tol)
75             break;
76     }
77     if(show_pbar)

```

```

78     std::fprintf(stderr, "\n");
79
80     // Write results back to raw uint8_t* buffer
81     for(int i = 0; i < n_pixels * 3; ++i) {
82         float val = std::max(0.0f, std::min(
83             current[i], 255.0f));
84         raw[i] = static_cast<uint8_t>(std::round(
85             val));
86     }

```

```

86     return MeanShiftResult{static_cast<int>(<
    iter_details.size()), total_shift_ms,
    iter_details};
87 }

```

Listing 6: Sequential brute-force Mean Shift function

7.2. The seq algorithm

```

1 MeanShiftResult meanShift(std::vector<uint8_t>&
    data, int width, float bandwidth,
2         int max_iter, float tol
    , bool show_pbar, KernelFn kernel) {
3     using clock = std::chrono::steady_clock;
4
5     if(!kernel) kernel = makeKernel("flat");
6
7     auto t_conv_start = clock::now();
8     std::vector<Pixel> current;
9     toPixels(data, current, width);
10    auto t_conv_end = clock::now();
11    double convert_ms = std::chrono::duration<
    double, std::milli>(t_conv_end - t_conv_start
    ).count();
12
13    const float bandwidth_sq = bandwidth *
    bandwidth;
14    const int n_pixels = static_cast<int>(current
    .size());
15
16    double total_shift_ms = 0.0;
17    int iter = 0;
18    std::vector<IterationInfo> iter_details;
19
20    // Hoist next outside the iteration loop -
    allocate once, reuse every iteration.
21    std::vector<Pixel> next(n_pixels);
22
23    for(; iter < max_iter; ++iter) {
24        float max_change = 0.0f;
25
26        auto t_shift_start = clock::now();
27
28        for(int i = 0; i < n_pixels; ++i) {
29            const Pixel& src = current[i];
30            float sum_r = 0.0f, sum_g = 0.0f,
    sum_b = 0.0f;
31            float weight_sum = 0.0f;
32
33            for(int j = 0; j < n_pixels; ++j) {
34                float w = kernel(squaredDistance(
    src, current[j]), bandwidth_sq);
35                if(w > 0.0f) {
36                    sum_r += w * current[j].r;
37                    sum_g += w * current[j].g;
38                    sum_b += w * current[j].b;
39                    weight_sum += w;
40                }
41            }
42
43            float change = 0.0f;
44            if(weight_sum > 0.0f) {
45                float new_r = sum_r / weight_sum;
46                float new_g = sum_g / weight_sum;
47                float new_b = sum_b / weight_sum;
48                change = std::max({std::abs(new_r
    - src.r),

```

```

49                std::abs(new_g
    - src.g),
50                std::abs(new_b
    - src.b)}});
51            next[i] = {new_r, new_g, new_b,
    src.x, src.y};
52        } else {
53            next[i] = src;
54        }
55
56        if(change > max_change)
57            max_change = change;
58    }
59
60    auto t_shift_end = clock::now();
61    double iter_ms = std::chrono::duration<
    double, std::milli>(t_shift_end -
    t_shift_start).count();
62    total_shift_ms += iter_ms;
63    iter_details.push_back({iter + 1, iter_ms
    , max_change});
64
65    current.swap(next);
66
67    if(show_pbar)
68        printProgressBar(iter + 1, max_iter,
    max_change);
69
70    if(max_change <= tol)
71        break;
72    }
73
74    if(show_pbar)
75        std::fprintf(stderr, "\n");
76
77    auto t_conv_out_start = clock::now();
78    fromPixels(current, data);
79    auto t_conv_out_end = clock::now();
80    convert_ms += std::chrono::duration<double,
    std::milli>(t_conv_out_end - t_conv_out_start
    ).count();
81
82    return MeanShiftResult{static_cast<int>(<
    iter_details.size()), total_shift_ms,
    convert_ms, iter_details};
83 }

```

Listing 7: Sequential Mean Shift function (seq)

7.3. The soa algorithm

```

1 MeanShiftResult meanShiftSoA(std::vector<uint8_t
    >& data, int width, float bandwidth,
2         int max_iter, float
    tol, bool show_pbar, KernelFn kernel) {
3     using clock = std::chrono::steady_clock;
4
5     if(!kernel) kernel = makeKernel("flat");
6
7     auto t_conv_start = clock::now();
8     std::vector<float> current;
9     convertToFloat(data, current);
10
11    ImageSoA soa;
12    convertToFloatSoA(current, soa, width);
13    auto t_conv_end = clock::now();

```

```

14 double convert_ms = std::chrono::duration<
double, std::milli>(t_conv_end - t_conv_start
).count();

15
16 const float bandwidth_sq = bandwidth *
bandwidth;
17 int n_pixels = soa.n;
18
19 double total_shift_ms = 0.0;
20 int iter = 0;
21 std::vector<IterationInfo> iter_details;
22
23 // Hoist next buffers outside the iteration
loop - allocate once, reuse every iteration.
24 std::vector<float> next_r(n_pixels);
25 std::vector<float> next_g(n_pixels);
26 std::vector<float> next_b(n_pixels);
27
28 for(; iter < max_iter; ++iter) {
29     float max_change = 0.0f;
30
31     auto t_shift_start = clock::now();
32
33     for(int i = 0; i < n_pixels; ++i) {
34         NeighborAccumulator acc;
35
36         for(int j = 0; j < n_pixels; ++j) {
37             float w = kernel(
squaredDistanceSoA(soa, i, j), bandwidth_sq);
38             if(w > 0.0f) {
39                 acc.sum_r += w * soa.r[j];
40                 acc.sum_g += w * soa.g[j];
41                 acc.sum_b += w * soa.b[j];
42                 acc.weight_sum += w;
43             }
44         }
45
46         float change = 0.0f;
47         if(acc.weight_sum > 0.0f) {
48             float new_r = acc.sum_r / acc.
weight_sum;
49             float new_g = acc.sum_g / acc.
weight_sum;
50             float new_b = acc.sum_b / acc.
weight_sum;
51
52             float dr = new_r - soa.r[i];
53             float dg = new_g - soa.g[i];
54             float db = new_b - soa.b[i];
55
56             change = std::max({std::abs(dr),
std::abs(dg), std::abs(db)});
57
58             next_r[i] = new_r;
59             next_g[i] = new_g;
60             next_b[i] = new_b;
61         } else {
62             next_r[i] = soa.r[i];
63             next_g[i] = soa.g[i];
64             next_b[i] = soa.b[i];
65         }
66
67         if(change > max_change)
68             max_change = change;
69     }
70
71     auto t_shift_end = clock::now();

```

```

72     double iter_ms = std::chrono::duration<
double, std::milli>(t_shift_end -
t_shift_start).count();
73     total_shift_ms += iter_ms;
74     iter_details.push_back({iter + 1, iter_ms
, max_change});
75
76     soa.r.swap(next_r);
77     soa.g.swap(next_g);
78     soa.b.swap(next_b);
79
80     if(show_pbar)
81         printProgressBar(iter + 1, max_iter,
max_change);
82
83     if(max_change <= tol)
84         break;
85 }
86
87 if(show_pbar)
88     std::fprintf(stderr, "\n");
89
90 auto t_conv_out_start = clock::now();
91 convertFromFloatSoA(soa, current);
92 convertFromFloat(current, data);
93 auto t_conv_out_end = clock::now();
94 convert_ms += std::chrono::duration<double,
std::milli>(t_conv_out_end - t_conv_out_start
).count();
95
96 return MeanShiftResult{static_cast<int>(
iter_details.size()), total_shift_ms,
convert_ms, iter_details};
97 }

```

Listing 8: SoA brute-force Mean Shift function

7.4. The omp algorithm

```

1 MeanShiftResult meanShiftOMP(std::vector<uint8_t
>& data, int width, float bandwidth,
2                                     int max_iter, float
tol, bool show_pbar, KernelFn kernel) {
3     using clock = std::chrono::steady_clock;
4
5     if(!kernel) kernel = makeKernel("flat");
6
7     auto t_conv_start = clock::now();
8     std::vector<Pixel> current;
9     toPixelsOMP(data, current, width);
10    auto t_conv_end = clock::now();
11    double convert_ms = std::chrono::duration<
double, std::milli>(t_conv_end - t_conv_start
).count();
12
13    const float bandwidth_sq = bandwidth *
bandwidth;
14    const int n_pixels = static_cast<int>(current
.size());
15
16    // Hoist next outside the iteration loop -
allocate once, reuse every iteration.
17    std::vector<Pixel> next(n_pixels);
18
19    double total_shift_ms = 0.0;
20    int iter = 0;

```



```

21 std::vector<IterationInfo> iter_details;
22
23 for(; iter < max_iter; ++iter) {
24     float max_change = 0.0f;
25
26     auto t_shift_start = clock::now();
27
28     #pragma omp parallel for schedule(static)
29     reduction(max:max_change)
30     for(int i = 0; i < n_pixels; ++i) {
31         const Pixel& src = current[i];
32         float sum_r = 0.0f, sum_g = 0.0f,
33         sum_b = 0.0f;
34         float weight_sum = 0.0f;
35
36         for(int j = 0; j < n_pixels; ++j) {
37             float w = kernel(
38                 squaredDistanceOMP(src, current[j]),
39                 bandwidth_sq);
40             if(w > 0.0f) {
41                 sum_r += w * current[j].r;
42                 sum_g += w * current[j].g;
43                 sum_b += w * current[j].b;
44                 weight_sum += w;
45             }
46         }
47
48         float change = 0.0f;
49         if(weight_sum > 0.0f) {
50             float new_r = sum_r / weight_sum;
51             float new_g = sum_g / weight_sum;
52             float new_b = sum_b / weight_sum;
53             change = std::max({std::abs(new_r
54                 - src.r),
55                 std::abs(new_g
56                 - src.g),
57                 std::abs(new_b
58                 - src.b)});
59             next[i] = {new_r, new_g, new_b,
60                 src.x, src.y};
61         } else {
62             next[i] = src;
63         }
64
65         max_change = std::max(max_change,
66             change);
67     }
68
69     auto t_shift_end = clock::now();
70     double iter_ms = std::chrono::duration<
71     double, std::milli>(t_shift_end -
72     t_shift_start).count();
73     total_shift_ms += iter_ms;
74     iter_details.push_back({iter + 1, iter_ms
75     , max_change});
76
77     current.swap(next);
78
79     if(show_pbar)
80         printProgressBar(iter + 1, max_iter,
81             max_change);
82
83     if(max_change <= tol)
84         break;
85 }
86
87 if(show_pbar)

```

```

75     std::fprintf(stderr, "\n");
76
77     auto t_conv_out_start = clock::now();
78     fromPixelsOMP(current, data);
79     auto t_conv_out_end = clock::now();
80     convert_ms += std::chrono::duration<double,
81     std::milli>(t_conv_out_end - t_conv_out_start
82     ).count();
83
84     return MeanShiftResult{static_cast<int>(
85     iter_details.size()), total_shift_ms,
86     convert_ms, iter_details};
87 }

```

Listing 9: Parallel brute-force Mean Shift function

7.5. The *omp_soa* algorithm

```

1 MeanShiftResult meanShiftSoAOMP(std::vector<
2     uint8_t>& data, int width, float bandwidth,
3     int max_iter,
4     float tol, bool show_pbar, KernelFn kernel) {
5     using clock = std::chrono::steady_clock;
6
7     if(!kernel) kernel = makeKernel("flat");
8
9     auto t_conv_start = clock::now();
10    std::vector<float> current;
11    convertToFloat(data, current);
12
13    ImageSoA soa;
14    convertToFloatSoA(current, soa, width);
15    auto t_conv_end = clock::now();
16    double convert_ms = std::chrono::duration<
17    double, std::milli>(t_conv_end - t_conv_start
18    ).count();
19
20    const float bandwidth_sq = bandwidth *
21    bandwidth;
22    int n_pixels = soa.n;
23
24    // Hoist next buffers outside the iteration
25    loop - allocate once, reuse every iteration.
26    std::vector<float> next_r(n_pixels);
27    std::vector<float> next_g(n_pixels);
28    std::vector<float> next_b(n_pixels);
29
30    double total_shift_ms = 0.0;
31    int iter = 0;
32    std::vector<IterationInfo> iter_details;
33
34    for(; iter < max_iter; ++iter) {
35        float max_change = 0.0f;
36
37        auto t_shift_start = clock::now();
38
39        #pragma omp parallel for schedule(static)
40        reduction(max:max_change)
41        for(int i = 0; i < n_pixels; ++i) {
42            NeighborAccumulator acc;
43
44            for(int j = 0; j < n_pixels; ++j) {
45                float w = kernel(
46                    squaredDistanceSoA(soa, i, j), bandwidth_sq);
47                if(w > 0.0f) {
48                    acc.sum_r += w * soa.r[j];
49

```

```

41         acc.sum_g += w * soa.g[j];
42         acc.sum_b += w * soa.b[j];
43         acc.weight_sum += w;
44     }
45 }
46
47 float change = 0.0f;
48 if(acc.weight_sum > 0.0f) {
49     float new_r = acc.sum_r / acc.
weight_sum;
50     float new_g = acc.sum_g / acc.
weight_sum;
51     float new_b = acc.sum_b / acc.
weight_sum;
52
53     float dr = new_r - soa.r[i];
54     float dg = new_g - soa.g[i];
55     float db = new_b - soa.b[i];
56
57     change = std::max({std::abs(dr),
std::abs(dg), std::abs(db)});
58
59     next_r[i] = new_r;
60     next_g[i] = new_g;
61     next_b[i] = new_b;
62 } else {
63     next_r[i] = soa.r[i];
64     next_g[i] = soa.g[i];
65     next_b[i] = soa.b[i];
66 }
67
68 if(change > max_change)
69     max_change = change;
70 }
71
72 auto t_shift_end = clock::now();
73 double iter_ms = std::chrono::duration<
double, std::milli>(t_shift_end -
t_shift_start).count();
74 total_shift_ms += iter_ms;
75 iter_details.push_back({iter + 1, iter_ms
, max_change});
76
77 soa.r.swap(next_r);
78 soa.g.swap(next_g);
79 soa.b.swap(next_b);
80
81 if(show_pbar)
82     printProgressBar(iter + 1, max_iter,
max_change);
83
84 if(max_change <= tol)
85     break;
86 }
87
88 if(show_pbar)
89     std::fprintf(stderr, "\n");
90
91 auto t_conv_out_start = clock::now();
92 convertFromFloatSoA(soa, current);
93 convertFromFloat(current, data);
94 auto t_conv_out_end = clock::now();
95 convert_ms += std::chrono::duration<double,
std::milli>(t_conv_out_end - t_conv_out_start
).count();
96
97 return MeanShiftResult{static_cast<int>(

```

```

iter_details.size()), total_shift_ms,
convert_ms, iter_details};
98 }

```

Listing 10: Parallel (SoA) brute-force Mean Shift function

```

1 // SoA pixel representation - separate channels
  for better parallelism
2 struct ImageSoA {
3     std::vector<float> r;
4     std::vector<float> g;
5     std::vector<float> b;
6     std::vector<float> x; // spatial column
  coordinate (fixed)
7     std::vector<float> y; // spatial row
  coordinate (fixed)
8     int n;
9 };
10
11 // SoA neighbor accumulator - uses float
  weight_sum to support weighted kernels
12 struct NeighborAccumulator {
13     float sum_r = 0.0f;
14     float sum_g = 0.0f;
15     float sum_b = 0.0f;
16     float weight_sum = 0.0f;
17 };

```

Listing 11: Structure of Arrays (SoA) representation and weighted neighbor accumulator for intermediate calculations

```

1 __global__ void meanShiftKernel(
2     const float* __restrict__ r_in, const float*
__restrict__ g_in,
3     const float* __restrict__ b_in, const float*
__restrict__ x_in,
4     const float* __restrict__ y_in,
5     float* r_out, float* g_out, float* b_out,
6     int n, float bw_sq, unsigned int*
d_max_change_bits)
7 {
8     extern __shared__ float sh[];
9     float* sh_r = sh;
10    float* sh_g = sh + blockDim.x;
11    float* sh_b = sh + 2 * blockDim.x;
12    float* sh_x = sh + 3 * blockDim.x;
13    float* sh_y = sh + 4 * blockDim.x;
14
15    const int i = static_cast<int>(blockIdx.x *
blockDim.x + threadIdx.x);
16    const int tx = static_cast<int>(threadIdx.x);
17    const int bs = static_cast<int>(blockDim.x);
18
19    float src_r = 0.f, src_g = 0.f, src_b = 0.f,
src_x = 0.f, src_y = 0.f;
20    const bool active = (i < n);
21    if(active) {
22        src_r = r_in[i]; src_g = g_in[i]; src_b =
b_in[i];
23        src_x = x_in[i]; src_y = y_in[i];
24    }
25
26    float sum_r = 0.f, sum_g = 0.f, sum_b = 0.f;
27    int count = 0;
28 }

```

```

29 for(int tile_start = 0; tile_start < n;
   tile_start += bs) {
30     int load_j = tile_start + tx;
31     if(load_j < n) {
32         sh_r[tx] = r_in[load_j];
33         sh_g[tx] = g_in[load_j];
34         sh_b[tx] = b_in[load_j];
35         sh_x[tx] = x_in[load_j];
36         sh_y[tx] = y_in[load_j];
37     }
38     __syncthreads();
39
40     if(active) {
41         int tile_end = min(bs, n - tile_start
42 );
43         for(int jj = 0; jj < tile_end; ++jj)
44         {
45             float dr = src_r - sh_r[jj];
46             float dg = src_g - sh_g[jj];
47             float db = src_b - sh_b[jj];
48             float dx = src_x - sh_x[jj];
49             float dy = src_y - sh_y[jj];
50             float sq = dr*dr + dg*dg + db*db
51 + dx*dx + dy*dy;
52             if(sq <= bw_sq) {
53                 sum_r += sh_r[jj];
54                 sum_g += sh_g[jj];
55                 sum_b += sh_b[jj];
56                 count++;
57             }
58         }
59         __syncthreads();
60
61         if(active) {
62             float new_r = (count > 0) ? sum_r /
63 static_cast<float>(count) : src_r;
64             float new_g = (count > 0) ? sum_g /
65 static_cast<float>(count) : src_g;
66             float new_b = (count > 0) ? sum_b /
67 static_cast<float>(count) : src_b;
68
69             float change = fmaxf(fabsf(new_r - src_r),
70 ,
71                               fmaxf(fabsf(new_g - src_g),
72                                     fabsf(new_b - src_b)))
73 );
74             r_out[i] = new_r;
75             g_out[i] = new_g;
76             b_out[i] = new_b;
77
78             if(!isnan(change))
79                 atomicMax(d_max_change_bits,
80 __float_as_uint(change));
81         }
82     }
83 }

```

Listing 12: CUDA kernel for mean shift using Structure of Arrays (SoA) representation

7.6. Cost model analysis

The execution time of each brute-force variant is modeled as $T = C \times n^2 \times \text{iter}$, where C is the average cost per pixel-pair evaluation. This cost aggregates:

- **memory accesses**, loading 3 floats (baseline, stride-3), 5 floats (seq, stride-5), or 5 unit-stride streams (soa, SIMD-vectorized),
- **ALU operations**, 5D squared distance (14 FLOP) + kernel comparison + conditional accumulation,
- **overhead**, loop control, integer division/modulo (baseline only), pipeline stalls.

Empirical values derived from benchmarks: $C_{\text{baseline}} \approx 1.98$, $C_{\text{seq}} \approx 1.75$, $C_{\text{soa}} \approx 0.68$ (where $X/Y/Z$ are in ns or cycles per pixel-pair). The ratio $C_{\text{baseline}}/C_{\text{soa}} \approx 2.93$ confirms that SoA’s SIMD vectorization processes multiple pixels per instruction, while the constant speedup across resolutions follows from the n^2 cancellation — the same analysis referenced in section 5.1.1.

7.7. Scaling profiling

Profiling was conducted via `perf stat` on two images (add res here) for both `omp` and `omp-soa` at thread count 1,2,4,8,12 and 24, with 3 runs per configuration. Measured events included cycles, instructions, L1 data cache loads and misses, and backend stalls.

Profiling confirms three findings. First, OpenMP adds zero instruction overhead (fig. 12) — the count is identical at every thread count. The SoA variant executes approximately 14% more instructions than AoS (229×10^9 vs 201×10^9), a fixed tax for managing five separate memory streams, but compensates with higher IPC (4.1 vs 2.9). Second, the SoA cache advantage persists across all thread counts (fig. 13): L1 miss rate is 1.7% vs 12.6% for AoS, with only mild degradation at 24T (1.85%). The absolute L1 miss count is virtually identical for both, confirming the sequential analysis in §5.1.1: the rate difference is a SIMD artifact. Third, backend stalls/cycle (fig. 15) are consistently lower for SoA (~ 3.5 vs ~ 5.1), and CPU cycles (fig. 14) are flat up to 12 physical cores but approximately double at 24 threads under SMT. The $\sim 45\%$ IPC drop at 24T is identical for both layouts, indicating a hardware-level SMT bottleneck rather than anything layout-dependent. In summary, the profiling data confirms that OpenMP parallelization introduces zero instruction overhead, that the SoA advantage (higher IPC, lower L1 miss rate, fewer backend stalls) persists across all thread counts, and that the only significant performance degradation occurs under SMT (24 threads), where hardware resource contention affects both layouts equally.

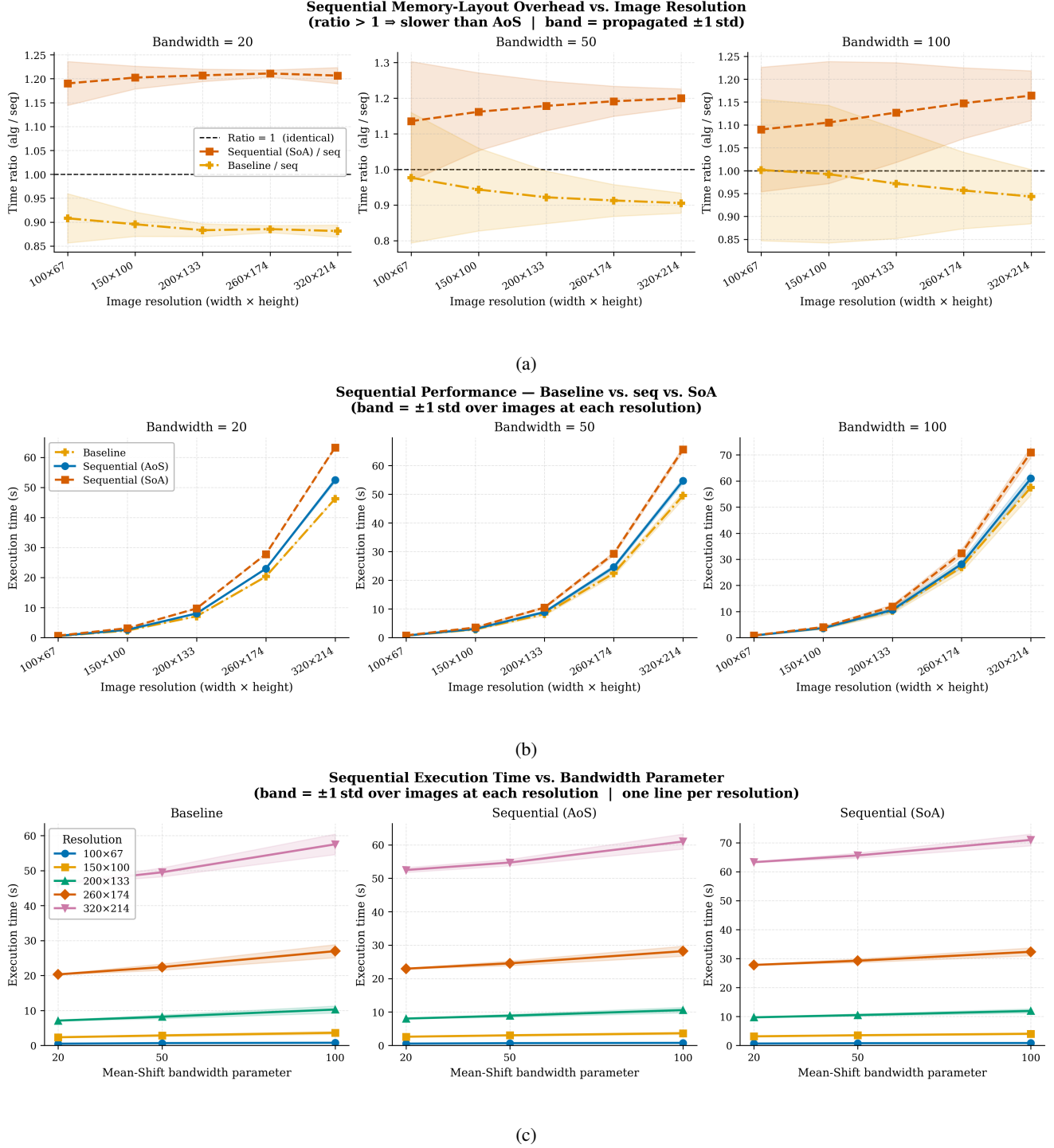


Figure 11: Sequential performance profiling of Mean Shift variants utilizing a dynamically swappable kernel on the AMD Ryzen 9900X. The abstraction overhead of this kernel design fundamentally inhibits compiler auto-vectorization. The resulting absence of SIMD acceleration dictates the anomalous performance characteristics observed across all three subfigures. **(a)** Memory layout overhead. The naive Baseline unexpectedly outperforms both pre-computed AoS and SoA layouts. Without SIMD registers to rapidly consume contiguous data, the memory-bound optimizations of SoA introduce detrimental scalar load latency. **(b)** Execution latency scaling. The Baseline maintains superior performance across spatial resolutions. In a purely scalar execution regime, the overhead of fetching disparate pre-computed coordinates from the L1 cache exceeds the latency of the Baseline’s on-the-fly arithmetic recomputation. **(c)** Bandwidth dependency. Expanding the spatial bandwidth introduces a distinct latency penalty. Lacking SIMD units to efficiently parallelize the $\mathcal{O}(N^2)$ neighbor evaluations, the pipeline becomes saturated by compute-bound scalar operations, exposing the direct arithmetic cost of the expanded search volume.

Table 5: Hardware counter summary for parallel variants (averaged across thread counts 1–12 physical cores; 24T shown separately for SMT comparison).

Metric	omp (1–12T)	omp_soa (1–12T)	omp_soa (24T)
IPC	~2.9	~4.1	~2.2
L1 miss rate	12.6%	1.7%	1.85%
Backend stalls/cyc	5.07	3.46	2.11
Instructions (norm)	1.000	1.000	1.001
Cycles (norm)	1.00	1.02	1.92

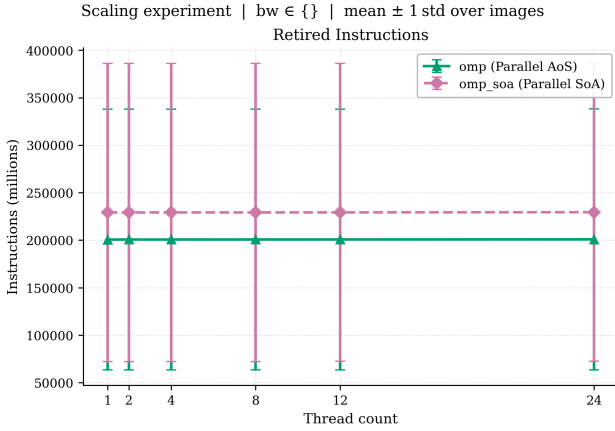


Figure 12: Total instructions per iteration for `omp` and `omp_soa` across thread counts. The count is flat across all threads (OpenMP adds zero overhead); SoA executes ~14% more instructions due to its five-stream memory layout.

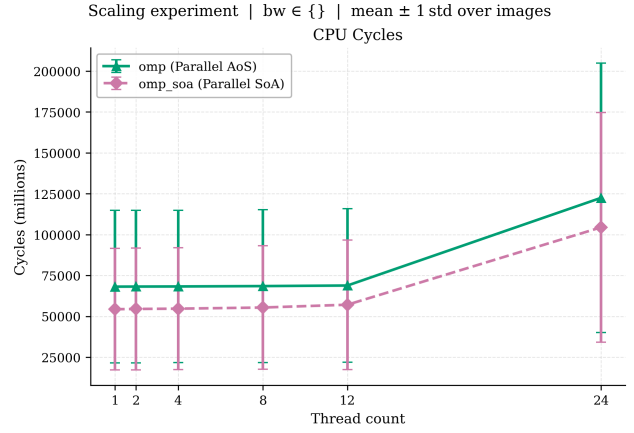


Figure 14: Total CPU cycles per iteration. Cycles are flat up to 12 physical cores (zero parallel overhead), then approximately double at 24 threads under SMT contention. The jump is similar for both variants, confirming a hardware-level effect.

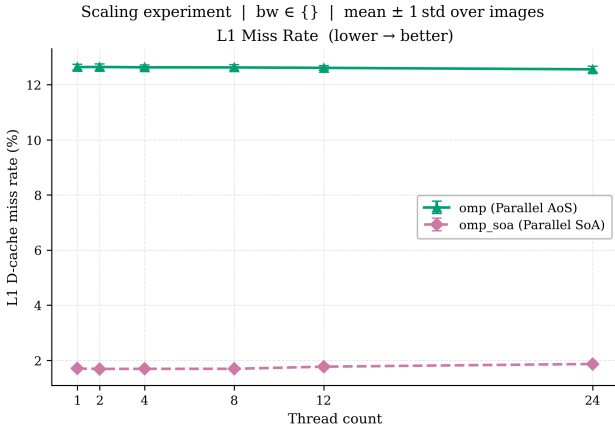


Figure 13: L1 data cache miss rate across thread counts. The SoA layout maintains ~1.7% vs ~12.6% for AoS at all thread counts, confirming the cache advantage is thread-count independent. Mild degradation at 24T (1.85%) suggests SMT cache-line bouncing.

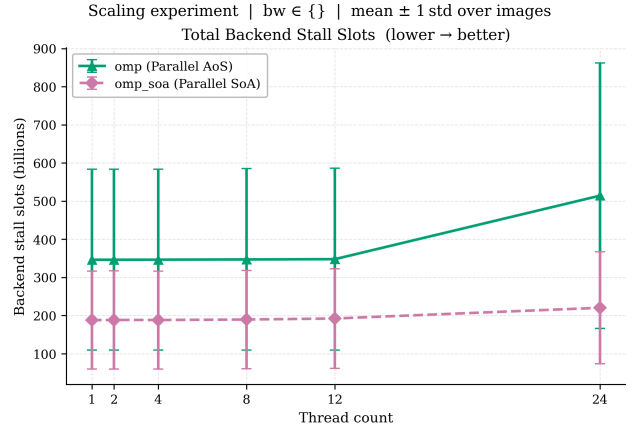


Figure 15: Backend stalls per cycle. SoA maintains consistently lower stall rates (~3.5 vs ~5.1), indicating the SIMD-vectorized inner loop keeps the execution pipeline better fed. The drop at 24T is a consequence of the IPC collapse (fewer instructions dispatched, fewer stalls accumulated).